Cambricon-R: A Fully Fused Accelerator for Real-Time Learning of Neural Scene Representation

Xinkai Song
SKLP,ICT,CAS
songxinkai@ict.ac.cn

Yuanbo Wen
SKLP,ICT,CAS
wenyuanbo@ict.ac.cn

Xing Hu*
SKLP,ICT,CAS SHIC
huxing@ict.ac.cn

Tianbo Liu
SKLP,ICT,CAS USTC
liutianbo@mail.ustc.edu.cn

Yifan Hao
SKLP,ICT,CAS
haoyifan@ict.ac.cn

Haoxuan Zhou
SKLP,ICT,CAS UCAS
zhouhaoxuan18@mails.ucas.ac.cn

Husheng Han
SKLP,ICT,CAS UCAS
hanhusheng20z@ict.ac.cn

Tian Zhi
SKLP,ICT,CAS
zhitian@ict.ac.cn

Zidong Du
SKLP,ICT,CAS SHIC
duzidong@ict.ac.cn

Wei Li
SKLP,ICT,CAS
liwei@ict.ac.cn

Rui Zhang
SKLP,ICT,CAS
zhangrui@ict.ac.cn

Chen Zhang
SJTU
chenzhang.sjtu@sjtu.edu.cn

Lin Gao
ICT,CAS
gaolin@ict.ac.cn

Qi Guo
SKLP,ICT,CAS
guoqi@ict.ac.cn

Tianshi Chen
Cambricon Technologies
tchen@cambricon.com

ABSTRACT

Neural scene representation (NSR) initiates a new methodology of encoding a 3D scene with neural networks by learning from dozens of photos taken from different camera positions. NSR not only achieves significant improvement in the quality of novel view synthesis and 3D reconstruction but also reduces the camera cost from the expensive laser cameras to the cheap color cameras on the shelf. However, performing 3D scene encoding using NSR is far from real-time due to the extremely low hardware utilization (only 0.22% utilization of hardware peak performance), which greatly limits its applications in real-time AR/VR interactions

In this paper, we propose Cambricon-R, a fully fused on-chip processing architecture for real-time NSR learning. Initially, by performing a thorough characterization of the computing model of NSR on a GPU, we find that the extremely low hardware utilization is mainly caused by the fragmentary stages and heavy irregular memory accesses. To address these issues, we propose Cambricon-R architecture with a novel fully-fused ray-based execution model to eliminate the computing and memory inefficiencies for real-time NSR learning. Concretely, Cambricon-R features a ray-level fused architecture that not only eliminates the intermediate memory traffics but also leverages the point sparsity in scenes to eliminate unnecessary computations. Additionally, a high throughput on-chip memory system based on Auto-Interpolation Bank Array (AIBA) is proposed to efficiently handle a large volume of irregular memory accesses.

We evaluate Cambricon-R on 12 commonly-used datasets with the state-of-the-art representative algorithm, instant-ngp. The result shows that Cambricon-R achieves PE utilization of 67.3%, on average. Compared to the state-of-the-art solution on A100 GPU, Cambricon-R achieves 373.8× speedup and 256.6× energy saving, on average. More importantly, it enables real-time NSR learning with **69.0 scenes per second**, on average.

CCS CONCEPTS

• **Computer systems organization** → **Architectures**; **Real-time systems**.

KEYWORDS

neural scene representation; hardware accelerator;

ACM Reference Format:
Xinkai Song, Yuanbo Wen, Xing Hu, Tianbo Liu, Yifan Hao, Haoxuan Zhou, Husheng Han, Tian Zhi, Zidong Du, Wei Li, Rui Zhang, Chen Zhang, Lin Gao, Qi Guo, and Tianshi Chen. 2023. Cambricon-R: A Fully Fused Accelerator for Real-Time Learning of Neural Scene Representation. In *56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23), October 28–November 01, 2023, Toronto, ON, Canada.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3613424.3614250

1 INTRODUCTION

Neural scene representation (NSR) and neural rendering have recently emerged as promising techniques to reconstruct realistic

*Corresponding Author.

MICRO '23, October 28–November 01, 2023, Toronto, ON, Canada

ACM ISBN 979-8-4007-0329-4/23/10…$15.00
https://doi.org/10.1145/3613424.3614250

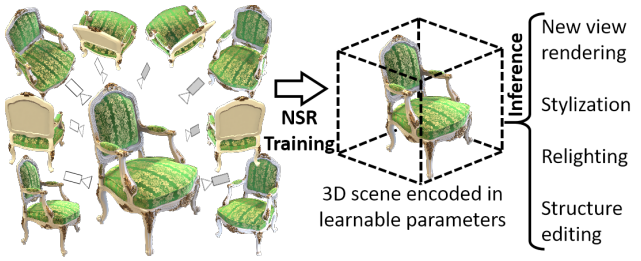


Figure 1: Training and inference of NSR.

complex 3D scenes with impressive quality. Compared with conventional graphic primitives and rendering methodologies [5, 10], NSR techniques eliminate the huge human efforts to construct precise 3D mesh models [1, 35, 36] and reduce the camera cost from the expensive laser cameras to the cheap color cameras on the shelf. Thus, NSR can boost productivity for augmented reality (AR) and virtual reality (VR) applications by automatically reconstructing and rendering realistic 3D scenes.

Figure 1 displays the two phases of NSR techniques. First, in the learning phase, NSR encodes the 3D scene implicitly into trainable parameters by learning from several photos taken from different spatial positions. Second, in the inference phase, the learned parameters can be used to implement many applications such as novel view rendering, stylization, relighting, and structure editing of the 3D scene encoded in these parameters. In this paper, we focus on accelerating the learning phase of NSR algorithms because learning NSR model incurs tremendous computing overhead, which is the key factor restricting the wide adoption of NSR techniques. The learning process takes several 2D photos as inputs and executes hundreds of forward-backward-update iterations of the following stages, as shown in Figure 2. 1) *Point Sampling* samples a set of points along a ray inside the range of the 3D scene (each ray is shot from the camera point through a pixel of an input photo). 2) *Point Encoding* encodes the coordinate and view direction of each sampled point into feature vectors. 3) *MLP* takes the encoded feature vector as inputs and then executes the MLP network to predict the emitted color and volume density for each sampling point. 4) *Loss* accumulates the colors and densities of sample points by ray integration along the ray corresponding to each image pixel, then calculates the loss between the predicted and ground-truth pixels. 5) *Parameters Updating* updates all the trainable parameters using the gradients learned from the loss.

NSR learning algorithms suffer from low performance on existing mainstream GPGPU platforms, which hinders their wide application. For AR/VR applications that require real-time interactions, there exists a ~ 321× performance gap between SOTA software NSR learning implementations and the real-time requirement. We noticed an extremely low hardware utilization of NSR learning algorithms on GPGPUs. For instance, the state-of-the-art NSR learning algorithm, instant-ngp [23], only achieves 0.22% of hardware utilization rate on an A100 GPU[1]. The reason lies in twofolds: 1) *fragmentary stages* and 2) *large volume of fine-grained memory accesses* of NSR learning algorithms.

[1]Hardware utilization rate = $NV \div RT \div PP$. NV indicates the number of operations for computing the valid points, RT indicates the runtime, PP indicates the peak performance of a device (312 Tflops for an A100 GPU).

Fragmentary stages: Compared to conventional neural networks, NSR kernels are much more fragmentary with a very low computation-to-memory traffic ratio. Batching strategies, typically adopted for improving hardware utilization in optimizing traditional DNN models, are not helpful for fragmentary stages. Contradictory to common sense, increasing the batch sizes not only contributes little to the data reuse but even generates a huge volume of intermediate data between two adjacent kernels, which greatly incurs significant off-chip memory traffic and deteriorates bandwidth efficiency in the following step. For example, the total volume of intermediate data accesses reaches 1.05 GB for a single iteration of training NSR, while the memory footprint is only around 570 MB. Hence, it is crucial to design a new computing model for boosting NSR computing efficiency.

The huge volume of fine-grained irregular memory accesses in the *Encoding* stage is a challenging problem for existing GPU platforms. As a consequence, the *Encoding* stage costs more than 29% of the total overhead of a training iteration (Figure 3), with huge off-chip memory access (Figure 4). Fine-grained subarray architectures may alleviate the problem, but introduces the tremendous bank conflicts and area overhead. This motivates us to design an on-chip memory system to support high-throughput fine-grained memory accesses.

To address these issues, we propose Cambricon-R architecture with a novel fully-fused ray-based execution model, to eliminate the computing and memory inefficiencies for real-time NSR learning. The novel ray-based execution model fully fuses the computing stages of all points in one ray, targeting the sweet-point of high computing parallelism and memory efficiency. Cambricon-R processes multiple rays simultaneously for high performance and eliminates the intermediate memory accesses inside one ray. Cambricon-R also leverages the early termination in ray marching to eliminate unnecessary computations with the insight that the background points are occluded by the foreground points and can be abandoned during NSR learning. Further, Cambricon-R includes a high throughput on-chip memory system based on Auto-Interpolation Bank array (AIBA) to efficiently handle the large volume of irregular memory accesses. Experimental results show that Cambricon-R achieves PE utilization of 67.3% and compared to the state-of-the-art solution on A100 GPU, Cambricon-R achieves up to **373.8× speedup** and **256.6× energy saving**. The contributions of this paper are summarized below.

- We perform a detailed characterization of neural scene representation algorithms, figuring out that the key issue is the low hardware utilization due to the fragmentary stages and irregular memory accesses.
- We propose a ray-based execution model and hardware pipeline to improve the computing parallelism and eliminate most of the off-chip memory accesses to increase hardware utilization. Following the ray-based execution flow, we implement software based on GPU which is 1.398× faster than the state-of-the-art solution.
- We propose Cambricon-R, the first fully fused on-chip processing accelerator with the key components of ray-based fused computing units and the AIBA-based on-chip memory

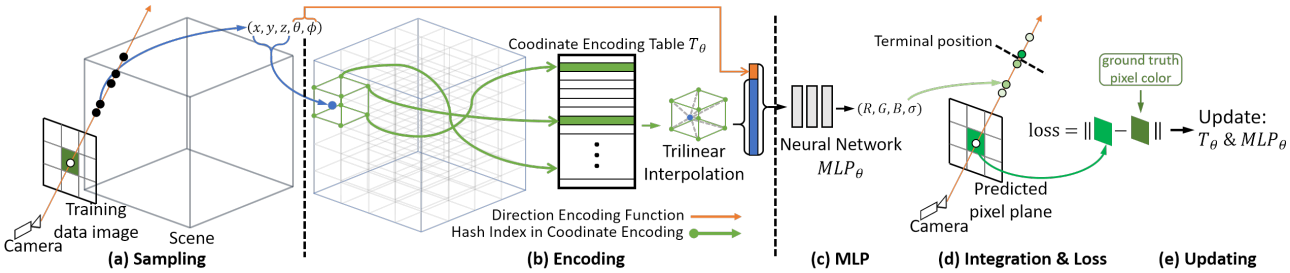


Figure 2: Computation of Neural Scene Representation.

systems that eliminate the memory inefficiency and pushes the NSR learning to the real-time level.

- We perform a thorough evaluation of Cambricon-R and the result shows that Cambricon-R achieves up to 373.8× speedup and 256.6× energy saving compared to the baseline solution.

2 NSR BASIS

In this section, we introduce the background and detailed algorithm of NSR, and discuss the application scenarios of NSR learning accelerators.

2.1 NSR Background

NSR is an emerging representation technique for encoding and representing 3D scenes. Traditional representation methods leverage meshes [6, 7, 27] or point clouds [3, 26] to build 3D scene models, which demands huge manual design efforts and expensive laser camera devices, and thus has a limited application scope. In recent years, NSR has made it possible to automatically learn a complex scene with impressive quality, which attracts a lot of attention for researchers in the field of computer vision and graphics. As shown in Figure 1, NSR can be divided into two phases. First, by learning from dozens of 2D input photos shot from different camera positions, the learning phase of NSR encodes the structure and color details of a 3D scene implicitly into learnable parameters in the encoding tables and the neural network weights. Then, in the inference phase, the learned neural representations for the 3D scene can be used to synthesize photos in new views and serve for broad scenarios such as rendering, reconstruction, stylization, relighting, and structure editing. The inference phase of NSR can be processed in real-time on modern GPUs. We focus on accelerating the learning phase of NSR algorithms in this paper.

The original NSR algorithm, NeRF [21], requires a huge amount of computations for the large MLPs and thus it is far from real-time. To mitigate this issue, Müller et al. propose instant-ngp [23], which turns a large part of MLP computations into feature vector look-up operations to achieve state-of-the-art performance for both learning and rendering while guaranteeing high-quality scene representations. However, the learning process of NSR still costs about seconds to train the parameters for a static 3D scene[2]. It cannot achieve real-time dynamic 3D scene representation.

[2]We choose the Instant-ngp as a representative of NSR learning algorithm in this paper. Instant-ngp reports the 5 seconds of minimum runtime for training a scene with acceptable quality.

2.2 The learning process of NSR

Figure 2 illustrates the learning process of NSR. The training data includes a set of photos and corresponding camera positions. The outputs of NSR are the weights of MLPs and encoding tables that are trained by minimizing the predicted and ground-truth color of pixels, Figure 2(d). Typically, the learning process of NSR involves hundreds of forward-backward-update iterations of the following stages:

1) Sampling: Given a photo and corresponding camera position, each pixel in the image plane is associated with a ray that starts from the camera viewpoint and passes through the pixel into the 3D scene. The algorithm then randomly samples points P along each ray, and outputs the corresponding 3D coordinates (x, y, z) and the 2D viewing direction (θ, ϕ) of the sampled points, shown in Figure 2(a).

Algorithm 1 Encoding Stage

Input: P[0:batchsize][0:5] // points
Input: T[0:16][0:2^{19}][0:2] // tables
Output: F[0:batchsize][0:48] // features
  for pid in range(batchsize) **do**
    $x, y, z \leftarrow P[pid][0:3]$
    $\theta, \phi \leftarrow P[pid][3:5]$
    for level in range(16) **do**
      $vertices[0:8] \leftarrow Surround(level, (x, y, z))$
      for vid in range(8) **do**
        $vtx[vid][level][0:2] \leftarrow$ **LookUp(vertices[vid],T[level])**
      end for
      $cooFea[level][0:2] \leftarrow TriIntpola(vtx[0:8][level][0:2])$
    end for
    $dirFea[0:16] \leftarrow SphericalEnc(\theta, \phi)$
    $F[pid][0:48] \leftarrow Concat(cooFea[0:16][0:2], dirFea[0:16])$
  end for

2) Encoding: For each point, the encoding stage takes the 5D tuple as input and outputs the encoded feature vector, as shown in Figure 2(b). We also provide a pseudo to describe the complex encoding computation as shown in Algorithm 1. The encoding includes two parts: the direction and the coordinate encoding. Since the direction encoding is computed by a simple function, we focus mainly on the complex coordinate encoding. The coordinate encoding uses 16 encoding tables with different levels of grid resolution[3]. At each level, the feature vectors of 8 grid vertices (blue) that surround the sampled point (x, y, z) are looked up from the corresponding table through a hash index function Equation 1. These 8 vectors are then

[3]Figure 2(b) only displays one level for simplicity

reduced as a feature vector of the point by trilinear interpolation. Finally, the output feature vector of a point is computed by concatenating the feature vectors of all 16 resolution levels and the direction encoding results.

$$hashIndex(x, y, z) = x\pi_0 + y\pi_1 + z\pi_2 \mod \#entries^4 \quad (1)$$

3) MLP: The encoded feature vector for each point is fed into two MLP networks (i.e., RGB network and density network) to predict an RGB color triplet $\mathbf{c}(r, g, b)$ and volume density σ. Typical configurations for both two MLP networks are extremely tiny, which is 32-64-64-3 and 32-64-16 for the two networks, respectively.
4) Loss: The Loss stage includes three phases. First, it computes the terminal position of each ray. Separated by the terminal position, the points near the camera are valid while the points away from the camera are occluded and will not be used in training. The terminal position is computed by accumulating the densities of the points near the camera until it reaches a threshold. Second, it performs ray integration to accumulate colors and densities of the valid points along a ray to predict the color of the corresponding pixel, as shown in Equation 2b. Third, it calculates the loss to measure the difference between the predicted color and ground-truth color for the rays in each training batch, as shown in Equation 2a.

$$\textbf{Loss} : \mathbf{c}(r, g, b), \sigma \rightarrow loss = \sum_{\mathbf{r}} \left\| \hat{C}(\mathbf{r}) - C_{g.t.}(\mathbf{r}) \right\|_2^2, \quad (2a)$$

$$\text{where } \hat{C}(\mathbf{r}) = \sum_{p} T_p (1 - \exp(-\sigma_p \delta_p)) \mathbf{c}_p \quad (2b)$$

Then, the backward processes of the MLP and the encoding stages are executed to compute the gradients of all trainable parameters.
5) Updating: During the NSR learning process, the trainable parameters such as MLP weights and the encoding table entries are updated by minimizing the former loss using the gradient descent algorithm. After every 16 iterations, a process called *the occupancy grid updating* is executed. This stage computes the binary densities of vertices in regular grids of the whole space. And the updated occupancy will be used in the sampling stage to avoid sampling points in the sparse space.

In this paper, we focus on the costly learning process of the state-of-the-art NSR algorithm, instant-ngp, and aim to realize real-time NSR learning.

3 NSR CHARACTERIZATION

In this section, we perform thorough analysis of the performance of NSR by running the instant-ngp algorithm using the tiny-cuda-nn framework on the A100 GPU. Although this is already the state-of-the-art solution for speeding up NSR training, there is still a $\sim 321\times$ performance gap towards real-time application requirements. In general, achieving real-time requires NSR learning for 60 scenes in one second [11, 34, 43], which means finishing a single scene under 16.7 ms. However, the state-of-the-art NSR learning algorithm costs at least 5 seconds. The profiling results show that the computing efficiency of A100 GPU for performing NSR learning is only 0.22%. We make an in-depth analysis and attribute the causes of the low hardware utilization rate to fragmentary stages and heavy fine-grained irregular encoding data accesses as follows.

3.1 Fragmentary stages

As shown in Figure 2, the learning process of NSR consists of five distinct computing stages: sampling, encoding, MLP, loss, and updating. We breakdown the runtime of the five stages on two typical datasets on A100, shown in Figure 3. Although MLP is the most compute-intensive stage, it only costs 17.33% and 16.88% of the total runtime for fox and ficus, respectively. In contrast, the other four stages dominate the total runtime, since they are completely different from the MLP stage and some of them are not suitable for GPU architecture, which leads to low computing efficiency.

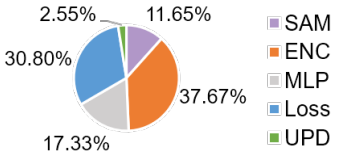
It is noticed that the loss stage takes a large portion of runtime in Figure 3 while the operation complexity is much lower than the encoding and MLP stages. After carefully reviewing the codes of the CUDA kernel, we found that the reason is twofold. First, only 32 thread blocks are launched for the loss kernel, which means that it uses at most 32 SMs (Streaming Multiprocessors) out of the total 108 SMs. Second, 32 threads in a warp process 32 different rays. The data for different rays are independent of each other. It results in a severe thread divergence in both the conditional branches and the discontinuous data addresses. Thus, GPU is inefficient when executing the loss stage.

Fragmentary stages with a large batch size (typically 262,144) also introduce a large number of intermediate data accesses, which further exacerbates the extremely low computing efficiency. In practice, these stages are implemented in different CUDA kernels because they require different thread schedules. For example, each thread in the MLP kernel is in charge of a certain block of a tensor for a series of sample points, while each thread in the loss kernel computes the ray integration for every single ray. Thus programming those stages into one fusion kernel is impractical. Besides, since large batches are introduced to benefit the computing parallelism and PE utilization rate, the size of the intermediate data transferred between two adjacent kernels are increasing accordingly with the large batching size. Concretely, the total volume of intermediate data accesses reaches 1.05 GB for a single iteration of training NSR, while the memory footprint is only around 570 MB. This significantly lowers computing efficiency.

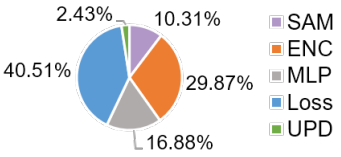
3.2 Heavy fine-grained irregular memory accesses in the encoding stage

The encoding stage is a performance bottleneck due to the large number of fine-grained irregular memory accesses. More than 29% of the total runtime of a training iteration is spent on the encoding stage according to Figure 3, with half of the total off-chip memory access happening in this stage according to Figure 4. Memory access in the encoding stage is challenging for the following three reasons. 1) *Fine-grained memory access:* Each entry in the encoding table typically consists of two FP16 numbers, which is eight times shorter than the 32-byte cache line in GPU; 2) *Irregular memory access:* The indexes of the entries of the surrounding vertexes are computed by the hash function, which considerably breaks the locality of memory addresses of neighboring vertexes; 3) *Heavy memory access requests:* There are typically 16 encoding tables with different grid

[4] $\pi_1 = 1$; $\pi_2 = 2654435761$; $\pi_3 = 805459861$; #entries = 2^{18}



(a) Fox



(b) Ficus

Figure 3: Runtime breakdown.

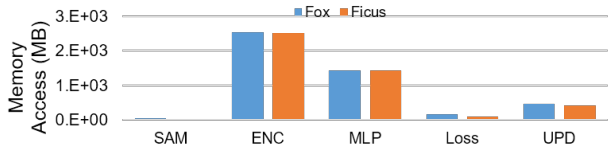


Figure 4: Off-chip memory access of different stages.

resolution level, and the entry indexes for each table are computed independently. And for one point, it needs to access 8 entries of the surrounding vertexes. Thus, 128 irregularly addressed 4-byte data fragments are required for every point. Moreover, the number of points to be processed in one training iteration is super large. The batchsize is typically 262,144 and in the forward process, it is even enlarged by up to 16 times because GPU does not utilize the early termination, see Section 4.3 for details.

Although instant-ngp is evaluated by a highly optimized NSR framework, tiny-cuda-nn, it is still insufficient for the encoding stage. It implements two main optimizations for the grid encoding kernel: the lower-bit precision and the level-by-level execution flow. The lower-bit precision reduces half of the encoding table data volume to no more than 32 MB (including the encoding data and their gradients) by using FP16 instead of FP32 data type. But it does not solve the problem of irregular memory access order caused by the hash function. The level-by-level optimization improves the data reuse on GPU caches. However, there is still a cache miss rate of 15.5% and a total of 2.5 GB off-chip memory access in one training iteration measured by the official tools from NVIDIA (Nsight-cli and Nsight-Compute). The reason is twofold. First, there are two types of input data, the encoding tables (up to 32 MB) and the input positions (up to 48 MB). The total size of these data is up to 80 MB, which is two times larger than the L2 cache of A100 GPU. Second, the irregular memory addresses break the cache locality. Thus, the heavy fine-grained irregular memory accesses become a bottleneck of the NSR performance.

In conclusion, the fragmentary stages and heavy fine-grained irregular memory access in the encoding stage cause the extremely low computing efficiency on the existing GPU platform and motivate us to design a brand-new accelerator to realize real-time NSR learning.

4 CAMBRICON-R

In this section, we propose Cambricon-R, a fully fused on-chip processing architecture for accelerating NSR learning. First, we propose the ray-based execution model instead of the pixel-based execution model commonly used in CNN image classification. Ray-based execution model largely improves intermediate data reuse with computing parallelism. Then, we design the Cambricon-R accelerator to implement a fully fused on-chip hardware pipeline for the NSR learning with the following design concepts: 1) Implementing hardware support for the on-chip ray-based execution model that eliminates 99% of the off-chip memory accesses. 2) Designing the early termination pipeline logic that avoids unnecessary computing in the forward process in observing that background pixels are blocked by the foreground pixels during rendering. 3) Proposing an area-efficient processing-on-the-way NoC (Network-on-Chip) design that solves the severe bank conflict problem caused by the huge number of fine-grained irregular memory accesses in the encoding stage.

4.1 Ray-based execution flow

GPU performs the NSR learning using a stage-based execution flow. Each stage is programmed into separate CUDA kernels. All kernels are executed with a large batch size, 262,144 valid points. This causes a huge memory footprint requirement of 571.1 MB. The memory footprint consists of two parts. One is the learnable parameters including the weights of MLPs (18 KB) and the entries in the encoding tables (32 MB). The other is the temporary data, including the training pixels, the sampled points, encoded features, the activations, and the gradients in the MLPs. The learnable parameters shared by all the points take only takes only 5.6% of the total memory footprint. The remaining memory footprint is spent on the temporary data whose sizes are proportional to the batch size.

This observation inspires us to reduce the granularity of data processing to reduce the memory footprint. The finest granularity of independent execution is the training on one ray because the loss function is computed using all valid points in a ray. Following this idea, we propose the ray-based execution flow to process all stages for one ray at a time. After the process of one ray is finished, the memory space that is occupied by the temporary data of this ray can be released and allocated to the next ray to start its processing. And The updating stage is executed after the last ray in an iteration of NSR is finished. As a result, the memory footprint for the temporary data is reduced to no more than 512 KB for a ray with 256 points. This is an acceptable size for the on-chip SRAM implementation.

4.2 Cambricon-R overall architecture

The architecture of Cambricon-R is designed based on the ray-based execution flow and we use the multi-core scheme to enlarge the parallelism and improve the up limit of performance. There are 128 MLP units and each of them can individually process the MLP training of one ray, including the MLP and loss stages. To save the on-chip memory spaces, Cambricon-R locates the shared trainable parameters such as the weights of MLP and encoding tables on the global buffer, and the temporary data such as the activations and gradients on the local buffer of each core. Moreover, we cache the

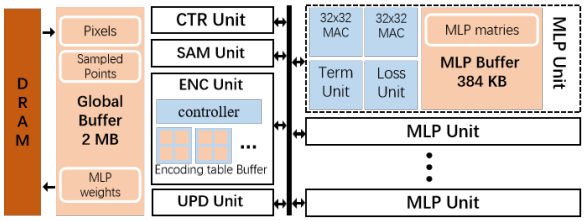


Figure 5: Cambricon-R Architecture.

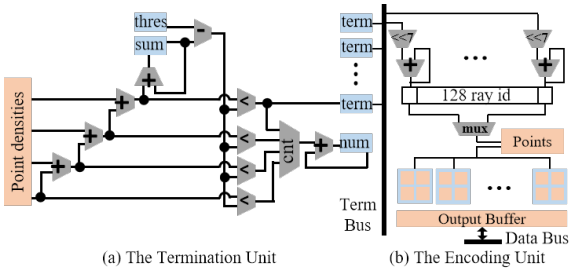


Figure 6: Early termination support.

weights of MLP on the local buffer to reduce global memory access since the size of MLP weights is only 18 KB. We will discuss the design and thorough dataflow of Cambricon-R below.

4.2.1 Overall architecture and dataflow. Figure 5 presents the overall architecture of the final design of Cambricon-R. It contains a sampling unit, an encoding unit, an updating unit, and 128 MLP units. The MLP unit contains two systolic arrays, a termination unit, and a loss unit. The loss unit only processes one ray at a time with all the data located in the local memory. According to our evaluation, the runtime of the loss unit is all covered by the MLP operations thanks to the hardware pipeline. The on-chip memory includes a global buffer, a lot of encoding buffers in the encoding unit, and 128 local buffers. The global buffer stores the raw training data (pixel rgbs), the sampled points, the weights of MLP, and the binary occupancy grid data. The encoding buffer stores the encoding tables and their gradients. The local buffer stores the data and their gradients of MLPs, the loss, and the accumulated weight gradients.

The dataflow on Cambricon-R is described as follows. At the beginning of NSR training, a batch of the raw training data is loaded from the DRAM into the global buffer. At the same time, the updating unit initializes the weights, stores them to the global buffer and then broadcast to all MLP units. Then, the sampling unit executes the sampling process and write the sampled points into the global buffer. After that, the encoding unit loads the sampled points and output the encoded points to the data bus which connects all MLP units. The MLP units then starts the forward process of the received points and the termination unit estimates the early termination once after the forward process of the received points is completed. The backward process in the MLP unit is started after the termination unit returns a valid termination signal. The backward process produces two types of data. The gradients of weights is accumulated and stored in the local buffer and the gradients of the inputs is only stored in the local buffer but not accumulated. This process is performed repeatedly after the total number of trained points exceeds the training batchsize.

At the end of each training iteration, the updating unit gathers the weight gradients in all MLP units, accumulates them and updates the weights in the global buffer. Meanwhile, the encoding unit gathers the gradients of inputs stored in all MLP units and updates the encoding tables. The above process of a training iteration is repeatedly executed for at least 250 iterations. After every 16 iterations, in the occupancy grid updating phase, the forward process of encoding and MLP stages are executed to compute the binary densities for the vertices of the grids in the whole space. And the result binary densities are stored in the global buffer. Finally, the weights and encoding tables are written to the DRAM and the training is finished.

4.3 Early Termination Optimization

The early termination mechanism is proposed by instant-ngp [23] to avoid the computation on the occluded points which are not used in training. [5] It cancels the computation on the background points according to the accumulated density of the foreground points in each ray. For example, for a ray with a total of 256 sampled points, if the accumulated density of the first 41 points exceeds the threshold, the left 215 points are useless in NSR learning process. However, GPU can only avoid the computation of invalid points in the backward process and waste lots of operations in the forward process. The reason is described as follows. The termination position is computed in the loss stage in GPU. Due to the stage-by-stage execution flow, GPU must complete the encoding and MLP stages of all sampled points before performing the loss stage. It results in a huge waste of 55.1% in the computation of encoding and MLP stages in the forward process.

Thanks to the fine-grained ray-based execution flow, early termination can be implemented during the forward process of the encoding and MLP stages in our design. First, we reduce the processing granularity from a whole ray to a group of 32 points for the encoding and MLP stages. Then, we place a termination unit in each MLP unit and perform early termination estimating immediately after the computation of a group of 32 points. Figure 6(a) displays the circuit in the termination unit to perform termination estimating efficiently. As a result, the waste of MLP forward computing on invalid points is limited to no more than 31 points for each ray.

However, canceling the encoding cost on invalid points is difficult due to the distance between 128 MLP units and the global encoding unit. If the encoding unit waits until the termination signal returns from each MLP unit and then starts encoding the next group, it will cause a lot of idle cycles in the MLP unit waiting for the next group of points due to the long latency of the encoding unit (detailed in the next subsection). To avoid these idle cycles, we propose to execute the encoding of the next group of points immediately without knowing the early termination signals from the MLP Units. As displayed in Figure 6(b), the termination signal returned to the encoding unit will not directly control the start or stop of the encoding processing. It terminates the encoding of the old ray by assigning a new ray to the corresponding MLP unit. When the MLP unit is computing the inference of the i-th group of points, the encoding unit is processing the (i+1)-th. If the termination

[5] Note that the early termination is not a change to the algorithm, so our design does not damage the correctness of computing results.

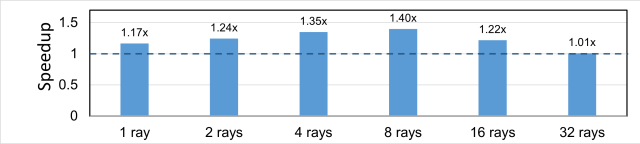


Figure 7: Design space exploration in CUDA optimization.

happened in the i-th group, the signal is returned to the encoding unit so that the (i+2)-th group of points will not be processed. The encoding of the (i+1)-th group of points is wasted. But this waste will not stall the MLP unit because after the termination the MLP unit starts to compute the loss and backward stages of the MLP. Thus, this solution efficiently maximizes the utilization of the MLP units.

4.3.1 GPU Optimization. **GPU implementation of the ray-based execution flow and early termination.** The ray-based execution flow and early termination can also be achieved through software approaches based on GPU. We heavily modified the CUDA kernels of tiny-cuda-nn (the fastest framework for NSR learning on GPU) by fusing all CUDA kernels in the forward phase into one large kernel, including the sampling, encoding, MLP, and termination detection functions.

The fused implementation brings two main advantages. First, it avoids the computation of invalid points through early termination. In the optimized program, the fused kernel executes the forward phase and detects termination for every 16 points in a ray iteratively. Second, it helps better utilize different types of hardware resources in parallel. In the optimized program, warps in different blocks can be executing different stages in parallel and the warp scheduler of GPU helps to fully utilize the idle hardware resources. As a comparison, in the baseline program, there is only one type of operation executed on each SM such that all warps are waiting for the same type of hardware resources. For example, the encoding stage heavily utilizes memory bandwidth while leaving the tensor cores idle. The MLP stage works in reverse. The optimized program avoids such congestion in hardware resources.

We also tried fusing kernels of both the forward and backward phases but found that fusing only the forward phase is the most efficient for GPU. We found that the raw ray-based execution flow of processing only one ray in each block is inefficient due to low parallelism. However, too many rays per block will cause waste in early termination and will reduce the number of blocks executed on each SM which can not well utilize the warp scheduler parallelism described above. We conduct design space exploration to find that 8 rays per block is the best trade-off with the highest performance, as shown in Figure 7. Finally, the optimized program achieves 1.398 × speedup compared with the original software, making it the new state-of-the-art solution based on GPU.

However, there is still a huge performance gap between the optimized GPU and the real-time requirement. The gap comes from two folds. First, GPU on-chip memory system does not match the encoding memory access pattern. The encoding stage requires an on-chip memory system with three characteristics: 1) large capacity to store all encoding tables; 2) global visibility to all SMs; 3) high throughput for accessing irregular 4-byte data. Specifically, L1/shared memory does not match the first two requirements. The L2 cache does not match the third requirement. The encoding stage in real-time NSR training requires a throughput of 4096 accesses of irregularly addressed 4-byte data per cycle (See Section 4.4.1) while A100's L2 cache has 80 slices each providing 64-byte data per cycle. Second, the lack of dedicated sample and loss circuits cost too much runtime on these lightweight functions (See Figure 3). These are both intrinsic reasons that can not be addressed by a software approach.

4.4 High Throughput Encoding Unit

Efficient memory accessing to the encoding tables is challenging even when the encoding tables are stored on-chip. In this subsection, we will introduce the design goal, challenges, baselines, and our design to solve this problem.

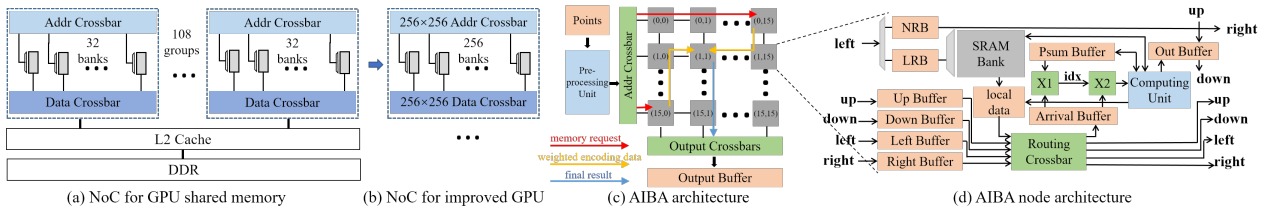
4.4.1 Design goal. According to our simulation, the real-time performance for NSR training requires a throughput of accessing 4096 irregularly addressed 4-byte words per cycle from the SRAM memory, assuming the frequency of the chip is 750 MHz.

To meet this requirement of memory throughput, the encoding tables must be stored in at least 4096 SRAM banks. However, designing the NoC (Network-on-Chip) connecting these SRAM banks with the subsequent processors is a challenging problem. The design objective of the NoC is to correctly transmit data between the SRAM banks and the output registers with **high throughput and low cost for the chip area**.

4.4.2 Challenges. This problem is challenging for two reasons. First, the high-radix connection required in this NoC is **expensive in the chip area**. Since memory access to different encoding tables is independent of each other, we can split the banks and output registers into 16 groups corresponding to 16 encoding tables. Because each group works independently, we will scale down the problem from 4096 to 256 banks in the later content of this paper. A large crossbar (256-to-256 fully connected NoC) is required inside each group to correctly move data from the banks to the right output locations. Second, **severe bank conflicts** drag down the throughput. Each memory access request is randomly distributed among 256 banks because the index of each vertex entry is computed through a hash function. As a result, some banks need to process multiple memory access requests while others process none. The throughput is dragged down because all banks need to wait for the busiest bank to complete its memory accesses.

These challenges are not well addressed by the NoC of current architectures. We choose three typical baselines to analyze their performance and propose our design below.

NN Accelerators: The on-chip memory organization and NoC of traditional NN accelerators are unsuitable for executing the encoding stage. The major reason is that the on-chip I/O interfaces of current NN accelerators are designed for high throughput access to large matrixes and vectors. They do not provide high throughput for fine-grained I/O interface. To access a 4-byte word, it needs to access a much longer width of data (32-128 bytes for typical NN accelerators) from the SRAM and then discard most of the data. Thus, most of the bandwidth of SRAMs is wasted.



(a) NoC for GPU shared memory (b) NoC for improved GPU (c) AIBA architecture (d) AIBA node architecture

Figure 8: NoC topologies. (a) displays the shared memory organization and NoC. (b) improves GPU by enlarging the bank group by 8 times to eliminate data movement between SMs. (c) shows the global NoC architecture of the AIBA, and the node architecture is displayed in (d).

GPU: GPU also fails to efficiently perform the encoding stage, although the shared memory in GPU is designed for high throughput fine-grained (4 bytes) memory access (see Table 1). The NoC of shared memory in GPU is displayed in Figure 8. In each SM, 32 banks provide a maximal throughput of 32 4-byte words per cycle. Memory accesses to the 32 banks are completed through two crossbars (a 32x32 address crossbar and a 32x32 data crossbar). However, in the encoding stage processing, each table has to be stored in 8 SMs to meet the requirement of a throughput of 256 words per cycle. It will cause massive data movement between SMs because the shared memory is only visible by the local processors in the same SM. And the only way for an SM to access data from the shared memory of another SM is through writing and reading to the global memory. As a result, the shared memory organization and the NoC in GPU is not suitable for encoding stage processing.

Table 1: Bank configurations

	A100(shared memory)	Cambricon-R
Number of banks	3456	4096
Bank size	5.125 KB	8 KB
Word width	4-byte	4-byte
Group size	32	256
Global visible	no	yes

Optimized GPU: This inspires us to improve the NoC of GPU and see what happened. We implement a straightforward improvement of GPU by enlarging the group size of SRAM banks from 32 to 256 to avoid data movement between SMs. As displayed in Figure 8(b), in each bank group, the improved GPU baseline uses two 256x256 crossbars for intra-group data movement: an address crossbar and a data crossbar. The connection width is 11 bits[6] and 32 bits for the address crossbar and the data crossbar, respectively. Thus, each bank group can efficiently process 256 independent memory accesses of a 4-byte word, which meets our requirement of memory access granularity and peak throughput for accessing to an encoding table.

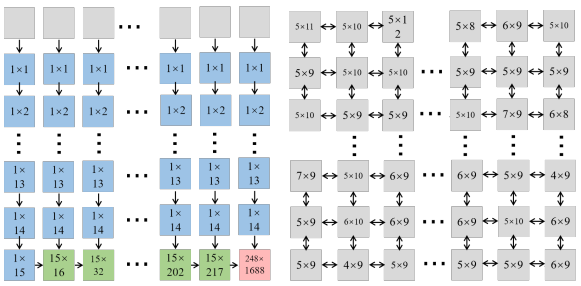
However, this solution has two shortcomings. First, the high-radix crossbar is expensive in area cost. In our experiment, the placing and routing of this NoC for one bank group failed after 3 weeks of running because of the massive wire crosses. We estimate the core area of the two 256x256 crossbars in Figure 8[b] by linearly scaling up the results reported in [41]. Under the 90 nm technology, the address crossbar and the data crossbar occupy the core area of at least $22mm^2$ and $64mm^2$, respectively. After scaling to 45nm technology node according to [31], the two crossbars occupy $30.68mm^2$ in total. However, we synthesize, place, and route the SRAM banks in a group using TSMC 45nm technology and get an area cost of $60.16mm^2$ in total. In conclusion, the area of the NoC in the improved GPU takes 50.9% of the area of SRAM banks.

Second, the problem of bank conflict is also unsolved in the improved GPU baseline. The 256 banks in an SM can only process one instruction at the same time. Thus the memory requests are irregularly distributed among all banks, leading to a bottleneck at the busiest bank and limiting performance. We conduct an experiment to evaluate the throughput of the improved GPU using real memory access requests in the NSR training. The result shows that the improved GPU achieves an average throughput of only 54 words per cycle for one group of 256 banks, which means only 21.1% utilization of the peak throughput of all SRAM banks.

4.4.3 Our Design. To alleviate the expensive area cost of the high-radix connections and the bank conflict problem, we propose an architecture called the Auto-Interpolation Bank Array (AIBA) to store the encoding tables and process the encoding computations. Figure 8(c) displays the overall architecture of AIBA. It uses a 2D-mesh NoC topology with each node connecting an SRAM bank. Because only the left column of nodes are connected with the address crossbar and only the bottom row of nodes are connected with the data crossbar. The area-expensive high-radix crossbars are avoided and the 2D-mesh topology only contains short connections between adjacent nodes and the cross points between the connections are dramatically reduced. The load/store instruction for the AIBA contains a group of 256 memory requests produced from 32 points (encoding of each point requires memory requests for 8 vertexes of the grid where the point belongs). The memory requests are sent into the bank array through the leftmost column of nodes. Then, each node transmits the requests to the node on the right side. The accessed data is transmitted to the node on the bottom side. The pre-processing unit is used to compute destination nodes, vertex addresses, and interpolation weights. The architecture of each node is displayed in Figure 8(d), it contains a routing crossbar, two request buffers for the memory access requests, a computing unit, and many other buffers for data routing, vertex interpolation, and result outputting. The detailed design and dataflow are described below.

Asynchronization control mode of banks: We solve the problem of bank conflict by changing the control mode of all banks from synchronization to asynchronization. For the improved GPU baseline, the 256 banks in a group are controlled synchronously and

[6]The typical capacity of each bank is 8KB (equal to 2048 words).



(a) The baseline dataflow (b) AIBA dataflow

Figure 9: Dataflow. The numbers indicate the max sizes of two vectors of data to be matched using a crossbar. In (a), different colors indicate different types of data, blue for matching the locally accessed data with the received data from the top side node, green for matching the received data from the top side node with the received data from the left side node, and red for the received data from the left side node and the unfinished results stored in the bottom-right node. In (b), the numbers indicate the matching sizes between the received data and the unfinished results.

process the same instruction for accessing 256 words at any time. The 256 words are not evenly distributed among 256 banks because of the irregularity of entry addresses. So, each bank processes a different number of memory access requests. As a result, the throughput of this bank group is limited by the busiest bank.

In asynchronization mode, all banks are controlled independently and the new memory access instruction is decoded and sent to all banks no longer waiting for the finish of the old instruction. Each bank has a request buffer to cache the conflicting requests which can be processed when there is no request arriving at this bank. This makes a balance between the busy cycles and the idle cycles for each bank. In this case, the overall throughput of the bank group can also be limited by the size of request buffers. When the request buffer of any bank is full, the decoding of the next instruction must be stalled until enough space is released to cache new requests. We conducted a simulation experiment to figure out the sufficient buffer size for each bank. The simulation assumes infinite buffer sizes for all banks so there is no stall for the instruction decoding. And it shows that the maximum number of requests stored in the buffer of each node is no more than 107 according to our simulation. All these buffers occupy $1.69mm^2$ of total area in 45 nm technology node, which is a negligible cost compared with the SRAM banks.

Interpolation inside the AIBA: The asynchronization control mode causes a problem in allocating the outputs of the bank group. Under the sync mode, in one cycle, the bank group outputs no more than 256 entries and they all belong to the same instruction, corresponding to 256 memory access requests. But under the async mode, the entries that are outputted in one cycle may belong to different instructions (up to 77 instructions, according to our simulation). As a result, each output needs to be allocated to one of the 19712[7] possible locations. It requires a 256×19712 crossbar switch to allocate 256 outputs to the right location, which is unpractical in implementing.

We address this challenge following the idea of completing the trilinear interpolation of each 8 vertexes inside the AIBA and then outputting the result entries of points. This can reduce the number of output entries by 8 times. To implement this idea, the interpolation weight and the point_id of each vertex are packed and transmitted with the memory requests. And we need to design a dataflow to decide where to compute the interpolation and data move path.

AIBA data flow vs. Naive dataflow: A straightforward design of dataflow is displayed in Figure 9(a). The data is first moved downward. After arriving at the bottom row of nodes, the data is then moved to the right side node. In each node, there are three sources of data. First, the data from the local SRAM. Second, the data from the top side node. Third, the data from the left side node (only for the bottom row of nodes). And the last two types of data may include the accessed raw data, the partial result of interpolation, and the final result of interpolation. Each node will first recognize the data that belong to the same output point by comparing the point_id of each pair of data from different sources, then perform the trilinear interpolation computation, and transmit the data and the results to the next node. Finally, all data and (partial) results arrive at the right-bottom node.

However, the cost for implementing this dataflow is too expensive. The implementation of this dataflow requires complex matching circuits in each node to recognize which pair of data belong to the same point. The area cost of the matching circuits grows with the lengths of the vectors to be matched. Figure 9(a) displays the max lengths of vectors to be matched in each node according to our simulation. Note that the number of unfinished results stored in the bottom-right node reaches 1688 and a 248×1688 crossbar is required, which is impractical in implementation.

We propose the AIBA dataflow to alleviate the area cost for the results matching before output. It is designed following the idea of distributing the up to 1688 unfinished results from only the red node of Figure 9(a) to all the 256 nodes. This dataflow is displayed in Figure 9(b) and the execution process of it is described as follows.

Figure 8(c) presents an example of memory access and interpolation operations for one sampled point using AIBA dataflow.

Pre-processing. The pre-processing unit computes the addresses, interpolation weights, and the shared destination node of the 8 vertices corresponding to a sampled point. The destination node is responsible for accumulating the weighted encoding vectors for interpolation (assuming it is node (1,1) in Figure 8(c)). The destination node is selected to balance the overhead among all nodes and minimize the total distance of data movement. These data are then packaged along with the index of the sampled points and sent into the nodes through the address crossbar and the row connections (red arrows in Figure 8(c))).

Loading and transmitting encoding data. After receiving the package, each node first loads the encoding data from the local SRAM and multiplies it by the interpolation weight. Then, it transmits the weighted encoding data toward its destination nodes through the routing crossbar along the yellow arrow in Figure 8(c).

Accumulating partial sum. After arriving at the destination node, the data package is matched with the cached partial sum using the crossbar X1 in the destination node. X1 outputs the index of the matched partial sum. And then crossbar X2 transmits and adds the weighted encoding data of vertices to the matched partial result.

[7] 77x256=19712, because each instruction contains 256 memory requests.

Outputting. If the partial sum has accumulated its 8 vertices, the interpolation operation is completed. And the result will be transmitted through the column connections (the blue arrow in Figure 8(c)). After arriving at the bottom nodes in AIBA, the results will be transmitted to the right addresses in the output buffer through the output crossbar. Finally, the MLP units can access the encoded data from the output buffers of all AIBAs.

Backward and Update. At the beginning of the backward process, the gradient of a sampled point is transmitted from the MLP units into the pre-processing units. The pre-processing unit first computes the addresses and interpolation weights of 8 vertices using the coordinates cached in the points buffer in Figure 8(c), then it multiplies the interpolation weights by the gradients of encoding results to obtain the gradients of each vertex. After that, the gradients of 8 vertices are transmitted to the right nodes and accumulated. Within each node's SRAM, the first 4KB is used to store the encoding data, while the latter 4KB is used to store the accumulated gradients of each encoding vector. When a node receives a gradient of encoding data, the node adds it to the accumulated gradient stored at the encoding data address + 4KB. At the end of each training iteration, each node updates all of the encoding vectors in its SRAM using the accumulated gradients.

Updating the occupancy grid. In processing the occupancy grid updating, the AIBA needs to execute the forward process of encoding stage and the updated occupancy grid is stored in the global buffer instead of in the AIBA SRAMs.

Hardware Implementation: Figure 9(b) displays the maximal sizes of the matching crossbars of X1 and X2 for all nodes according to our simulation. We choose 8x13 as the final size of matching crossbar X1 and X2 in each node so that all data in the arrival buffer will be processed immediately. The total area cost of these matching crossbars is 1.07 mm^2 in 45 nm. Besides, the data movement toward the destination node causes a lot of traffic through the Routing Crossbar in each node. According to our simulation, a 20x20 crossbar is enough for the routing crossbar to transmit data without stalling.

After the interpolation is completed in each node, the results are transmitted vertically to the bottom nodes, and each bottom node outputs two words per cycle and each node has an output buffer for caching the congested outputs. According to our simulation, the 32 words outputted from the bank array in each cycle belong to no more than 28 instructions. We use a hierarchical crossbar connection to allocate the output data. The first 32x32 crossbar splits data into 32 buffers according to the instructions. Then in each buffer, a 32x32 crossbar is used to allocate data to the right position. These 33 crossbars take a total area of 0.91mm^2 in 45 nm, which is acceptable compared with the area of SRAM banks.

In summary, AIBA solves the bank conflict problem by asynchronously controlling all the SRAM banks. Meanwhile, we optimize the area cost by designing a novel dataflow to avoid the use of high-radix connections due to data congestion. It achieves a throughput of encoding 29.1 points per cycle which means 91% of the peak throughput for 256 SRAM banks, while the baseline only achieves 21%. The total area cost of an AIBA is 71.25mm^2 in 45 nm. All 256 SRAM banks take 60.16mm^2, and the extra area used for the buffers, small crossbars, and connections is 11.09mm^2 in total. In conclusion, the AIBA NoC only costs 18.4% of extra area compared with the total area of all SRAM banks and achieves 91% of the peak throughput, while the baseline NoC costs 50.9% of the area of all SRAM banks and only achieves 21% of the peak throughput.

5 EXPERIMENTAL EVALUATION

5.1 Methodology

5.1.1 Hardware Implementation. We use both the hardware synthesizing tools and a simulator to measure the performance and energy consumption of Cambricon-R. We implement the RTL design of Cambricon-R with Verilog. We synthesize, place, and route the key units of Cambricon-R using Design Compiler and IC Compiler with TSMC 45nm technology node. Since the baseline NVIDIA A100 is in 7nm technology node, the results of area and power are then scaled to 7nm technology node according to [31]. Such a scaled comparison manner is commonly used in research papers [15] [19]. The energy of memory access for HBM 2.0 is estimated at 3.9 pJ/bit [25].

For performance evaluation, We build a cycle-accurate simulator in Python to measure the execution time, the number of DRAM access, and PE utilization. In this simulator, the number of cycles for memory access to DRAM is measured by invoking a warped interface of Ramulator [12]. Our simulator is running on 8 servers with AMD EPYC 7742 64-Core processor in parallel. We do not use the RTL simulator to evaluate the performance because the NSR tasks have too many operations and the full scale of our architecture is too large. To validate the accuracy of this simulator, we compare it with the RTL simulator by running a small algorithm configuration with a small hardware configuration. The validation result shows that the evaluated cycles of our simulator are less than the results of the RTL simulator by no more than 6.72%.

5.1.2 Experimental Configurations. We implement Cambricon-R using 128 MLP units, each consisting of two 32×32 MAC (Multiply-Accumulate) arrays and a 384 KB local buffer. The global buffer size is 2 MB. The encoding unit has 16 AIBAs. Each AIBA consists of 256 nodes with 256 8KB SRAM banks. Cambricon-R works with a frequency of 750 MHz. For a fair comparison, we use HBM2 with 1,555 GB/s bandwidth as off-chip memory, which is the same as our baseline GPU. Hardware characteristics of Cambricon-R are listed in Table 2. To highlight the area efficiency of AIBA, we separately list the areas of other circuits in AIBA except for the SRAM. The Encoding Unit (Other) in Table 2 includes the cost of pre-processing units, computing units, crossbars, buffers, and so on.

Table 2: Area and Power

	Area (mm^2)	%	Power (W)	%
Sampling Unit	0.10	0.06	0.24	0.11
Encoding Unit (SRAM)	56.77	34.56	71.60	34.65
Encoding Unit (Other)	10.47	6.38	16.45	7.96
MLP Units	93.32	56.82	113.80	55.08
Global Buffer	3.55	2.16	4.48	2.17
Other	0.03	0.02	0.06	0.03
Total	164.24	100	206.62	100

5.1.3 Benchmark. We evaluate 12 datasets from 6 scenes, including 2 dynamic scenes and 4 static scenes. Two dynamic scenes are jumpingjacks and standup. And each dynamic scene provides 4

datasets with 4 different poses of the same person captured at 4 different time points. The contents of datasets from the same scene are very similar. To increase the diversity of benchmarks, we select 4 more datasets from 4 static scenes, including Fox, Ficus, Hotdog, and Mic. The Fox dataset is from a real scene and the other three datasets are from synthetic scenes. Moreover, we use the typical algorithm hyper-parameters. The architectures of the MLPs are (32-64-16) and (32-64-64-3) for the density and color networks, respectively. There are 16 encoding tables and each table has 256k entries with a feature vector length of two FP16 values. The training batchsize is 262,144 valid points, and the total training iteration is set to 250.

5.1.4 Baselines. We use two baselines for comparison, a GPU and an NN accelerator. The GPU we use is NVIDIA A100-PCIE-40GB, which features a peak performance of 312 TFLOPs (FP16 dense tensor performance), a total on-chip memory of 87.25 MB, and 40 GB off-chip HBM2 memory with a bandwidth of 1,555 GB/s. We use the state-of-the-art software: the tiny-cuda-nn framework[22]. Tiny-cuda-nn is a highly optimized NSR framework for NVIDIA GPU and is invoked by the instant-ngp GitHub project. We use the latest version of the codes on GitHub till the paper submission deadline (commit: 89fe4166) and did not change the source code. The optimizations of tiny-cuda-nn are all used in our baseline evaluation. Notably, there are two versions of CUDA kernels for the MLP stage. We tried both kernels and selected the fast one (the FullyFusedMLP) as the GPU baseline in our paper.

The baseline of NN accelerator works as a stronger baseline to help evaluate the incremental breakdown. We use a TPU-like architecture and implement it with a cycle-accurate python simulator. We optimize the NN accelerator baseline with our ray-based execution flow and fully fused hardware pipeline. Otherwise, the huge volume of DRAM accesses will make the comparison meaningless. Besides, we use small systolic arrays to fit the small size of MLP in NSR. We add dedicated circuits for the sample, loss, and update functions so that these operations do not need to return to the CPU. The sizes of on-chip buffers for different stages are the same as Cambricon-R. However, the early termination and the AIBA are missing from the NN accelerator baseline. Furthermore, the word width for accessing SRAMs is 32 bytes because NN accelerators are commonly designed for vector and matrix operations and do not provide high throughput for fine-grained on-chip memory access. This baseline is called NNA-opt in this paper.

For a fair comparison, we list the hardware specifications for Cambricon-R and the baselines in Table 3.

Table 3: Hardware Specifications

	GPU-A100	NNA-opt	Cambricon-R
Number of cores	108	128	128
FP16 Peak Performance	312 TFLOPs	384 TFLOPs	384 TFLOPs
On-chip Memory	87.25 MB	82 MB	82 MB
Memory Bandwidth	1,555 GB/s	1,555 GB/s	1,555 GB/s

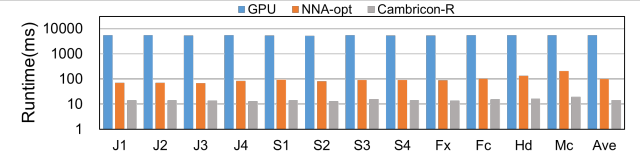


Figure 10: Performance comparison.

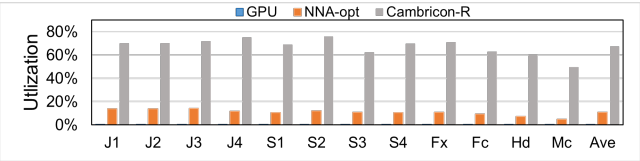


Figure 11: Comparison of the utilization of processing elements.

5.2 Evaluation Results

We compare Cambricon-R with the baselines in terms of performance, energy consumption, DRAM access, and PE utilization. After that, we conduct an ablation study to clarify the effectiveness of the early termination and the AIBA. Note that our solution does not change the computation of NSR algorithm, so the results should be the same as baseline. The average PSNR on these benchmarks is 29.14 with an acceptable image rendering quality.

5.2.1 Performance. Figure 10 presents the runtime of the whole training process which includes 250 training iterations for 12 benchmarks running on Cambricon-R and the baselines. J1-J4 represent for the four frames in Jumpingjacks, S1-S4 for the Standup, Fx for Fox, Fc for Ficus, Hd for Hotdog, and Mc for Mic. Results show that Cambricon-R achieves 373.8× and 6.73× speedup over GPU and the optimized NN accelerator on average, respectively. Note that the Cambricon-R achieves real-time with a throughput of 69.0 scenes per second. The optimized NN accelerator also achieves a speedup of 54.9× over GPU due to that the ray-based execution flow eliminates most of the DRAM access volume. Note that the performance for different scenes looks similar because our target algorithm is a training task which a fixed number of iterations and a fixed batchsize.

Figure 11 presents the PE utilization of Cambricon-R compared with the baselines on 12 benchmarks. It shows that Cambricon-R successfully addresses the low utilization problem of GPU by improving the average PE utilization from 0.22% to 67.3%. It can be noticed that when running on Cambricon-R, the utilization of different datasets varies in a large range from 49.7% to 75.73%, while the utilization is stable at 0.22% 0.23% when running on GPU. This is due to that the overall density of spaces of different scenes vary in a large range. During the stable period of training, the average valid points in each ray are quite different for all scenes. The Fox dataset is from a real scene with the highest complex, so the average number of valid points in a ray is around 78. While the other datasets are all from synthetic scenes with simple space structures. For example, the Mic dataset only has 16 valid points in each ray on average. It is only half of the execution granularity (32 points) of our early termination execution pipeline. Because Cambricon-R is executed in a ray-by-ray flow, there is some performance waste for these datasets with simple scene structures. For GPU, it executes in a

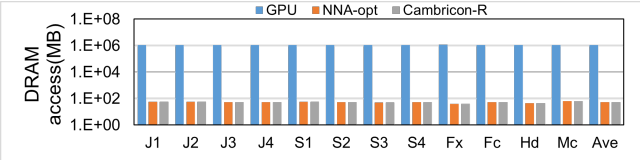


Figure 12: DRAM access comparison.

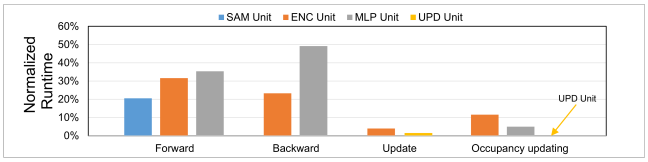


Figure 13: Runtime breakdown.

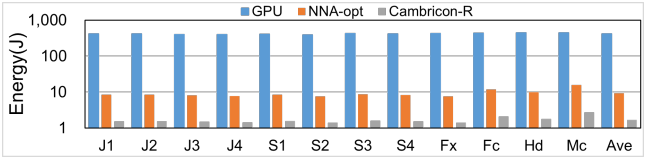


Figure 14: Energy consumption comparison.

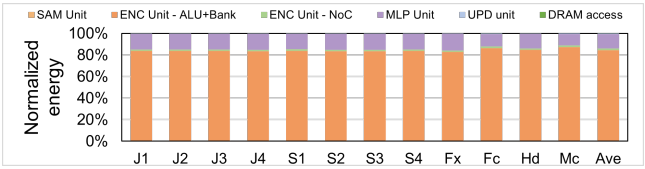


Figure 15: Energy consumption breakdown.

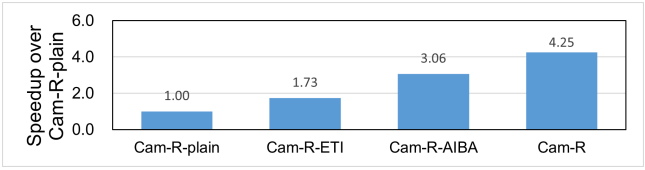


Figure 16: Ablation study on performance.

stage-by-stage flow and the total number of points in the whole batch is fixed (262,144). So, the number of points in every single ray will not influence the performance.

Figure 12 presents the comparison of Cambricon-R and the baselines in DRAM access. It shows that Cambricon-R reduces 99.995% the DRAM access. Cambricon-R only accesses 50.98 MB of data from or to DRAM on average. It is mainly because the fully fused Cambricon-R stores all the intermediate data on-chip and eliminates the redundant DRAM access of the intermediate data between different stages on GPU and only remains necessary DRAM access such as raw training data and the trained parameters. This also contributes to the extremely high utilization of processing elements of Cambricon-R displayed in Figure 11.

Figure 13 presents the runtime breakdown between different units in different stages. Each normalized runtime is relative to the total runtime of one iteration. Since the computation of different units in the same stage is pipelined, the total runtime of one stage is close to the runtime of the most time-consuming unit. Forward stage, backward stage, update stage, and occupancy updating stage account for 35.3%, 49.2%, 4.0%, and 11.5% of the total runtime, respectively. In both the forward and backward stages, the bottleneck lies in the MLP units. In the update stage, the bottleneck is the update of encoding tables. In the forward stage, although implementing the early termination, there are still no more than 31 invalid points may be executed for each ray, as explained in Section 4.3. Thus, the runtime for encoding in the backward stage is slightly shorter than that in the forward stage. And the reason behind the increased time consumption of the MLP in the backward stage is that MLP backward involves more computation per sampled point compared to MLP forward. In the occupancy updating stage, MLP units only compute the density network. Thus encoding units takes more time than MLP units.

5.2.2 Energy. We report the overall energy consumption of Cambricon-R and the baselines for processing 250 iterations of training on the 12 benchmarks as shown in Figure 14. The result shows that compared with GPU, Cambricon-R reduces the energy consumption by 256.6× on average. Note that the energy consumption of GPU is computed by multiplying the runtime and the average power reported by the nvidia-smi profiler tool. The optimized NN accelerator consumes 5.50× more energy than Cambricon-R on average. The reason is twofold. First, it wastes some operations on the invalid points due to the lack of early termination. Second, the 32-byte word width of the on-chip memory in the NN accelerator causes energy waste when processing a fine-grained 4-byte width memory request.

We further breakdown the energy consumption of Cambricon-R and display the result in Figure 15. The energy consumption is breakdown carefully into 5 parts, the energy consumed in the sampling unit, the encoding unit, the MLP units, and the off-chip memory access. And to evaluate the energy efficiency of NoC in AIBA, we further breakdown energy consumption in the encoding unit into two parts, the bank array and the NoC.

It should be noticed that the energy consumed in the off-chip memory access accounts for only 0.08% of the total energy consumption on average, indicating that most of the off-chip memory traffic are removed for Cambricon-R. The largest part of energy consumption is the memory access to the encoding tables, because of the huge amount of memory access to the encoding tables. The energy consumed in the NoC of the AIBA takes only 1.5% of the total energy consumption. This proves that the energy overhead of the NoC is negligible.

5.2.3 Optimization Analysis. To clarify the effectiveness of our design, we conduct an ablation study of the two optimizations including the early termination in inference (ETI) and the AIBA. We conduct the ablation study for Cambricon-R-plain, Cambricon-R-plain with ETI, Cambricon-R-plain with AIBA, and Cambricon-R. Figure 16 demonstrates the speedup of each solution over the Cambricon-R-plain. Compared with the Cambricon-R-plain baseline, the combination of optimizations of the ETI and the AIBA achieves an overall speedup of 4.25× on average. The ETI optimization improves the performance solely by 1.73× indicating that ETI effectively saves the computations on the inference of invalid points. The AIBA optimization improves the performance solely by 3.06×, which is achieved by the improvement in the throughput

in accessing the encoding tables. This ablation study proves the effectiveness of our key optimizations of the ETI and the AIBA.

6 RELATED WORK

We review the related works on neural scene representation algorithms and hardware accelerators for neural networks.

Neural scene representation. Neural scene representation is a promising technique and can benefit many applications, such as scene labeling and understanding [42], object category modeling [2, 39], depth estimation [37], model reconstruction [24, 36], free-viewpoint video [14, 16, 18, 29], pose estimation [9, 20, 32, 33], relighting [30, 38, 40] and so on. The process of NSR is both energy- and time-consuming. Thus, a lot of research work has been carried out for algorithm optimization. Liu *et al.* make the rendering process more than 10 times faster than NeRF by organizing the scene into a sparse voxel octree [17]. Hedman *et al.* proposed SNeRG to realize real-time rendering by using sparse voxel grid representation [8]. Müller *et al.* designed a multi-resolution hash table of trainable feature vectors to enable the use of a smaller and more efficient MLP model without sacrificing quality, and it significantly reduces the training costs to only 5 seconds [23].

Hardware accelerators. [13] proposes an accelerator for neural rendering pipeline. However, Cambricon-R targets different NSR tasks of training instead rendering. There are many accelerator works designed for neural networks. Many accelerators are proposed to optimize various neural network models. Geng *et al.* designed AWB-GCN architecture to address the issue of extremely large and unbalanced real-world graphs for accelerating GCN inference [4]. Robson *et al.* proposed ProSE, which consists of custom heterogeneous systolic arrays and special functions for protein and NLP-related tasks [28]. Our approach is distinct as we propose a fully fused on-chip processing architecture with a novel ray-based execution model based on our characterization of NSR learning to achieve real-time performance.

7 CONCLUSION

In this paper, we propose Cambricon-R, the first accelerator to push the NSR algorithm towards real-time application. Cambricon-R is a fully fused on-chip processing architecture with a novel ray-based execution model that eliminates more than 99.995% of the off-chip memory access. To improve the efficiency of Cambricon-R, we further propose the early termination in the inference process to avoid the waste of computations on invalid points in the forward process and improve the performances by 2×. We also propose the Auto-Interpolation Bank Array to largely alleviate the throughput waste due to the bank conflict and improve the utilization of throughput of bank arrays from 21% to 91%. Experimental results demonstrate that compared to the state-of-the-art solution on NVIDIA A100 GPU, Cambricon-R achieves 373.8× speedup and 256.6× energy saving. More importantly, it enables real-time 3D scene representation with 69.0 scenes per second.

ACKNOWLEDGMENTS

This work is partially supported by the National Key R&D Program of China (under Grant 2022YFB4501601), the NSF of China (under Grants U22A2028, 62222214, 62002338, U20A20227, 62102399, U19B2019), CAS Project for Young Scientists in Basic Research (YSBR-029) and Youth Innovation Promotion Association CAS. We thank Jin Li from Tsinghua University and Boxin Zhang from Harbin Institute of Technology for their help in GPU evaluation.

REFERENCES

[1] 2018. Neural scene representation and rendering. *Science* 360, 6394 (2018), 1204–1210.
[2] Hendrik Baatz, Jonathan Granskog, Marios Papas, Fabrice Rousselle, and Jan Novák. 2021. NeRF-Tex: Neural Reflectance Field Textures. In *EGSR*.
[3] Haoqiang Fan, Hao Su, and Leonidas J Guibas. 2017. A point set generation network for 3d object reconstruction from a single image. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 605–613.
[4] Tong Geng, Ang Li, Runbin Shi, Chunshu Wu, Tianqi Wang, Yanfei Li, Pouya Haghi, Antonino Tumeo, Shuai Che, Steve Reinhardt, et al. 2020. AWB-GCN: A graph convolutional network accelerator with runtime workload rebalancing. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 922–936.
[5] Toshiya Hachisuka and Henrik Jensen. 2010. Parallel progressive photon mapping on GPUs. *SIGGRAPH Sketches* (01 2010). https://doi.org/10.1145/1899950.1900004
[6] Peter Hedman, Julien Philip, True Price, Jan-Michael Frahm, George Drettakis, and Gabriel Brostow. 2018. Deep blending for free-viewpoint image-based rendering. *ACM Transactions on Graphics (TOG)* 37, 6 (2018), 1–15.
[7] Peter Hedman, Tobias Ritschel, George Drettakis, and Gabriel Brostow. 2016. Scalable inside-out image-based rendering. *ACM Transactions on Graphics (TOG)* 35, 6 (2016), 1–11.
[8] Peter Hedman, Pratul P. Srinivasan, Ben Mildenhall, Jonathan T. Barron, and Paul E. Debevec. 2021. Baking Neural Radiance Fields for Real-Time View Synthesis. *ArXiv* abs/2103.14645 (2021).
[9] Yoonwoo Jeong, Seokjun Ahn, Christopher Bongsoo Choy, Anima Anandkumar, Minsu Cho, and Jaesik Park. 2021. Self-Calibrating Neural Radiance Fields. *ArXiv* abs/2108.13826 (2021).
[10] James T. Kajiya. 1986. The Rendering Equation *(SIGGRAPH '86)*. Association for Computing Machinery, New York, NY, USA, 143–150. https://doi.org/10.1145/15922.15902
[11] Yongwoo Kim, Jae-Seok Choi, and Munchurl Kim. 2018. 2X super-resolution hardware using edge-orientation-based linear mapping for real-time 4K UHD 60 fps video applications. *IEEE Transactions on Circuits and Systems II: Express Briefs* 65, 9 (2018), 1274–1278.
[12] Yoongu Kim, Weikun Yang, and Onur Mutlu. 2015. Ramulator: A fast and extensible DRAM simulator. *IEEE Computer architecture letters* 15, 1 (2015), 45–49.
[13] Junseo Lee, Kwanseok Choi, Jungi Lee, Seokwon Lee, Joonho Whangbo, and Jaewoong Sim. 2023. NeuRex: A Case for Neural Rendering Acceleration. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–13.
[14] Tianye Li, Miroslava Slavcheva, Michael Zollhoefer, Simon Green, Christoph Lassner, Changil Kim, Tanner Schmidt, S. Lovegrove, Michael Goesele, and Zhaoyang Lv. 2021. Neural 3D Video Synthesis. *ArXiv* abs/2103.02597 (2021).
[15] Zheng Li, Soroush Ghodrati, Amir Yazdanbakhsh, Hadi Esmaeilzadeh, and Mingu Kang. 2022. Accelerating attention through gradient-based learned runtime pruning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 902–915.
[16] Zhengqi Li, Simon Niklaus, Noah Snavely, and Oliver Wang. 2021. Neural Scene Flow Fields for Space-Time View Synthesis of Dynamic Scenes. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2021), 6494–6504.
[17] Lingjie Liu, Jiatao Gu, Kyaw Zaw Lin, Tat-Seng Chua, and Christian Theobalt. 2020. Neural Sparse Voxel Fields. *NeurIPS* (2020).
[18] Stephen Lombardi, Tomas Simon, Jason M. Saragih, Gabriel Schwartz, Andreas M. Lehrmann, and Yaser Sheikh. 2019. Neural volumes. *ACM Transactions on Graphics (TOG)* 38 (2019), 1 – 14.
[19] Nika Mansouri Ghiasi, Jisung Park, Harun Mustafa, Jeremie Kim, Ataberk Olgun, Arvid Gollwitzer, Damla Senol Cali, Can Firtina, Haiyu Mao, Nour Almadhoun Alserr, et al. 2022. GenStore: a high-performance in-storage processing system for genome sequence analysis. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 635–654.
[20] Quan Meng, Anpei Chen, Haimin Luo, Minye Wu, Hao Su, Lan Xu, Xuming He, and Jingyi Yu. 2021. GNeRF: GAN-based Neural Radiance Field without Posed Camera. *ArXiv* abs/2103.15606 (2021).
[21] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. 2020. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. In *ECCV*.
[22] Thomas Müller. 2021. Tiny CUDA Neural Network Framework. https://github.com/nvlabs/tiny-cuda-nn.

[23] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. 2022. Instant Neural Graphics Primitives with a Multiresolution Hash Encoding. *ACM Trans. Graph.* 41, 4, Article 102 (July 2022), 15 pages. https://doi.org/10.1145/3528223.3530127

[24] Michael Oechsle, Songyou Peng, and Andreas Geiger. 2021. UNISURF: Unifying Neural Implicit Surfaces and Radiance Fields for Multi-View Reconstruction. *ArXiv* abs/2104.10078 (2021).

[25] Mike O'Connor, Niladrish Chatterjee, Donghyuk Lee, John Wilson, Aditya Agrawal, Stephen W Keckler, and William J Dally. 2017. Fine-grained DRAM: Energy-efficient DRAM for extreme bandwidth systems. In *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 41–54.

[26] Songyou Peng, Michael Niemeyer, Lars Mescheder, Marc Pollefeys, and Andreas Geiger. 2020. Convolutional occupancy networks. In *European Conference on Computer Vision*. Springer, 523–540.

[27] Gernot Riegler and Vladlen Koltun. 2020. Free view synthesis. In *European Conference on Computer Vision*. Springer, 623–640.

[28] Eyes Robson, Ceyu Xu, and Lisa Wu Wills. 2022. ProSE: the architecture and design of a protein discovery engine. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 655–668.

[29] Aliaksandra Shysheya, Egor Zakharov, Kara-Ali Aliev, Renat Bashirov, Egor Burkov, Karim Iskakov, Aleksei Ivakhnenko, Yury Malkov, I. Pasechnik, Dmitry Ulyanov, Alexander Vakhitov, and Victor S. Lempitsky. 2019. Textured Neural Avatars. *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2019), 2382–2392.

[30] Pratul P. Srinivasan, Boyang Deng, Xiuming Zhang, Matthew Tancik, Ben Mildenhall, and Jonathan T. Barron. 2021. NeRV: Neural Reflectance and Visibility Fields for Relighting and View Synthesis. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2021), 7491–7500.

[31] Aaron Stillmaker and Bevan Baas. 2017. Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm. *Integration* 58 (2017), 74–81.

[32] Shih-Yang Su, Frank Yu, Michael Zollhoefer, and Helge Rhodin. 2021. A-NeRF: Surface-free Human 3D Pose Refinement via Neural Rendering. *ArXiv* abs/2102.06199 (2021).

[33] Edgar Sucar, Shikun Liu, Joseph Ortiz, and Andrew J. Davison. 2021. iMAP: Implicit Mapping and Positioning in Real-Time. *ArXiv* abs/2103.12352 (2021).

[34] Amr Suleiman and Vivienne Sze. 2014. Energy-efficient HOG-based object detection at 1080HD 60 fps with multi-scale support. In *2014 IEEE Workshop on Signal Processing Systems (SiPS)*. IEEE, 1–6.

[35] Ayush Tewari, Ohad Fried, Justus Thies, Vincent Sitzmann, Stephen Lombardi, Kalyan Sunkavalli, Ricardo Martin-Brualla, Tomas Simon, Jason Saragih, and Matthias and Niener. 2020. State of the Art on Neural Rendering. *Computer Graphics Forum* 39, 2 (2020), 701–727.

[36] Peng Wang, Lingjie Liu, Yuan Liu, Christian Theobalt, Taku Komura, and Wenping Wang. 2021. NeuS: Learning Neural Implicit Surfaces by Volume Rendering for Multi-view Reconstruction. *ArXiv* abs/2106.10689 (2021).

[37] Yi Wei, Shaohui Liu, Yongming Rao, Wang Zhao, Jiwen Lu, and Jie Zhou. 2021. NerfingMVS: Guided Optimization of Neural Radiance Fields for Indoor Multi-view Stereo. *ArXiv* abs/2109.01129 (2021).

[38] Suttisak Wizadwongsa, Pakkapon Phongthawee, Jiraphon Yenphraphai, and Supasorn Suwajanakorn. 2021. NeX: Real-time View Synthesis with Neural Basis Expansion. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2021), 8530–8539.

[39] Christopher Xie, Keunhong Park, Ricardo Martin-Brualla, and Matthew A. Brown. 2021. FiG-NeRF: Figure-Ground Neural Radiance Fields for 3D Object Category Modelling. *ArXiv* abs/2104.08418 (2021).

[40] Xiuming Zhang, Pratul P. Srinivasan, Boyang Deng, Paul E. Debevec, William T. Freeman, and Jonathan T. Barron. 2021. NeRFactor: Neural Factorization of Shape and Reflectance Under an Unknown Illumination. *ArXiv* abs/2106.01970 (2021).

[41] Ying Ping Zhang, Taikyeong Jeong, Fei Chen, Haiping Wu, Ronny Nitzsche, and Guang R Gao. 2006. A study of the on-chip interconnection network for the IBM Cyclops64 multi-core architecture. In *Proceedings 20th IEEE International Parallel & Distributed Processing Symposium*. IEEE, 10–pp.

[42] Shuaifeng Zhi, Tristan Laidlow, Stefan Leutenegger, and Andrew J. Davison. 2021. In-Place Scene Labelling and Understanding with Implicit Scene Representation. *ArXiv* abs/2103.15875 (2021).

[43] Dajiang Zhou, Jinjia Zhou, Xun He, Jiayi Zhu, Ji Kong, Peilin Liu, and Satoshi Goto. 2011. A 530 Mpixels/s 4096x2160@ 60fps H. 264/AVC high profile video decoder chip. *IEEE Journal of Solid-State Circuits* 46, 4 (2011), 777–788.